String vs StringBuilder vs StringBuffer

Why it matters

This is asked in almost every Java interview and it has an easy answer everyone gives ("StringBuilder is faster, StringBuffer is synchronised") and a harder one that shows you understand *why*. It also has a genuinely important practical consequence: string concatenation in a loop is one of the most common real performance bugs in enterprise Java, and it turns a linear job into a quadratic one.

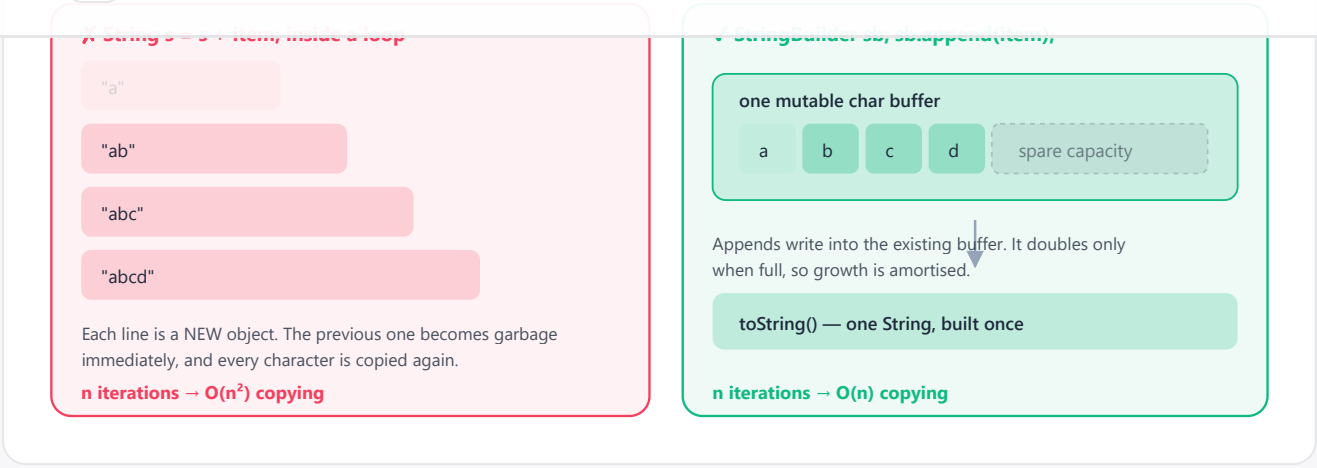
The concept, simply

	`String`	`StringBuilder`	`StringBuffer`
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	**No**	Yes (synchronised)
Speed	Slow for repeated edits	**Fastest**	Slower than Builder
Since	1.0	Java 5	1.0
Use when	Fixed text, keys, sharing	**Almost always**	Practically never

The honest summary: use `String` for values, `StringBuilder` when you are building text, and `StringBuffer` essentially never.

How it works

The performance difference is not a constant factor — it is a change in complexity class:



Concatenation in a loop copies every character accumulated so far, on every iteration. A StringBuilder writes into one buffer that doubles only when full.

With `String`, iteration *i* must allocate a new object and copy all *i* characters accumulated so far. Summing over *n* iterations gives O(n²) copying, plus *n* objects of garbage. With `StringBuilder`, each append writes into an existing `byte[]`; when it fills, the array doubles, so the total copying across all growth is amortised O(n).

Why `StringBuffer` is obsolete is the part worth articulating. It synchronises every method — `append`, `insert`, `length`, all of them. That costs something even uncontended, but the deeper problem is that per-method synchronisation is almost never the synchronisation you actually need. A builder used by one thread does not need it; a builder genuinely shared across threads needs the whole *sequence* of appends to be atomic, which locking each call individually does not provide. So `StringBuffer` pays for safety in the case where it is unnecessary and fails to deliver it in the case where it would matter. `StringBuilder` was added in Java 5 as the unsynchronised version, and in practice a builder is a local variable — the textbook case for needing no synchronisation at all.

Code examples

The bug, and its fix:

JAVA

Copy

```java
// ✗ O(n²). At 100,000 items this can take minutes.
String result = "";
```

```
for (String item : items) {
    ","; // new String allocated and fully copied,
}

// ✓ O(n). Milliseconds for the same input.
StringBuilder sb = new StringBuilder();
for (String item : items) {
    sb.append(item).append(',');
}
String result = sb.toString();

// ✓ Cleaner still when you are joining — no manual trailing-comma handlir
String result = String.join(",", items);
String result = items.stream().collect(Collectors.joining(", ", "[", "]"))
```

Where `+` is perfectly fine — and the distinction matters, because "never use +" is wrong advice:

JAVA Copy

```
// ✓ Fine. A single expression: javac compiles it to an invokedynamic call
//   that sizes the result and fills it in one pass (Java 9+).
String message = "User " + name + " logged in at " + timestamp;

// ✓ Fine. Compile-time constant — folded by javac into a single literal.
String url = "https://" + HOST + "/api";     // when HOST is a static fina

// ✗ Not fine. A LOOP: each iteration is a separate expression that must
//   produce a complete new String, so the optimisation cannot help.
for ( ... ) { s += x; }
```

Presizing, the small optimisation worth knowing when the length is predictable:

JAVA Copy

```
StringBuilder sb = new StringBuilder(1024);   // avoids repeated array gro
// Default capacity is 16 characters; each overflow allocates a new array
// roughly double the size and copies. Presizing skips that entirely when
// have a reasonable estimate.
```

```
    line.setLength(0);                          // clear, keep the capacity
    line.append(r.id()).append('\t').append(r.name());
    writer.write(line.toString());
}
```

And the reason `StringBuilder` should stay local:

JAVA Copy

```
// StringBuilder is NOT thread-safe. Shared across threads, appends interl
// and can corrupt the internal length/array pair — you can even get
// ArrayIndexOutOfBoundsException from inside append().
//
// The fix is not StringBuffer. It is: give each thread its own builder,
// or collect the pieces and join once.
String result = items.parallelStream().collect(Collectors.joining(","));
```

Under the hood

Both builders extend the package-private `AbstractStringBuilder` and share their implementation entirely; `StringBuffer` simply overrides each method with a `synchronized` version that calls `super`. That is the whole difference — same algorithm, one lock.

Both also got **compact strings** in Java 9, storing a `byte[]` with a coder flag rather than a `char[]`. There is a subtlety here worth knowing: if a builder holding Latin-1 content receives a character that does not fit, it must *inflate* — reallocate as UTF-16 and copy everything. Appending one emoji to a large ASCII builder is therefore unexpectedly expensive. Rarely decisive, but a genuinely modern detail.

The **Java 9** `invokedynamic` **change** to string concatenation is the piece that dates most interview answers. Older material says every `+` compiles into a `StringBuilder` — true up to Java 8. Since Java 9, `javac` emits an `invokedynamic` bound to `StringConcatFactory`, which generates a specialised method at run time that

computes the exact length and fills the result in a single pass. It is faster than the old builder-based expansion, and it is why "the compiler turns + into StringBuilder anyway" is no longer accurate. The loop case is unaffected, because the problem there is the repeated production of complete intermediate strings, not the code generated for any one expression.

Common mistakes

- **Concatenating with `+=` in a loop.** The headline bug. O(n²).
- **Concluding that `+` is always bad.** A single expression is fine and more readable than a builder.
- **Reaching for `StringBuffer` "to be safe".** It pays for a lock you do not need and does not give atomicity across a sequence anyway.
- **Sharing one `StringBuilder` across threads.** It is not thread-safe; give each thread its own or join at the end.
- **Building a string just to log it.** `log.debug("x=" + x)` concatenates even when debug is off. Use the parameterised form `log.debug("x={}", x)`.
- **Hand-rolling a join.** `String.join` or `Collectors.joining` handle separators and edge cases for you.
- **Ignoring initial capacity for large builds.** The default is 16 characters.

Interview lens

> **Q: "Difference between `String`, `StringBuilder` and `StringBuffer`?"**
> A: `String` is immutable, so every edit allocates a new object. `StringBuilder` is a mutable buffer and is not synchronised — the default choice. `StringBuffer` is the same thing with every method synchronised; it predates `StringBuilder` and is effectively legacy, because a builder is almost always a local variable that no other thread can see.

> **Q: "Why is concatenation in a loop bad?"**
> A: Because strings are immutable, iteration *i* allocates a new object and copies all

doubles when full, so the total work is amortised O(n).

Q: "Does the compiler not optimise `+` into a `StringBuilder` anyway?"
A: It did up to Java 8. Since Java 9 it compiles to an `invokedynamic` call into `StringConcatFactory`, which generates a method that sizes the result exactly and fills it in one pass — better than the old expansion. Either way it only helps a single expression. In a loop each iteration is its own expression producing a complete new string, so no code generation can rescue it.

Q: "When would you actually use `StringBuffer`?"
A: Realistically, only when maintaining old code that already uses it. If I genuinely had a buffer shared across threads I would not want per-method locking anyway — I would need the whole append sequence to be atomic, which `StringBuffer` does not give me. Far better to give each thread its own builder and join the results.

Q: "How would you join a list into a comma-separated string?"
A: `String.join(",", list)` for the simple case, or `Collectors.joining(", ", "[", "]")` in a stream when I need a prefix and suffix. Both use a builder internally and handle the trailing-separator problem that hand-rolled loops always get wrong.

Q: "What is `StringBuilder`'s default capacity?"
A: 16 characters, or the initial string's length plus 16. When it fills, the array grows to roughly double and everything is copied — so for a large known build, passing an initial capacity avoids several rounds of that.

- `String` immutable · `StringBuilder` mutable, **not** synchronised · `StringBuffer` mutable, synchronised, legacy.
- **Default to** `StringBuilder`; a builder is almost always a local variable.
- Loop with `+=` → **O(n²)**. Builder → amortised O(n).
- A **single** `+` expression is fine — Java 9+ compiles it via `invokedynamic`.
- Joining? `String.join` / `Collectors.joining`.
- Default capacity **16** — presize for large builds; `setLength(0)` to reuse.
- Logging: `log.debug("x={}", x)`, never `"x=" + x`.
- Both builders share `AbstractStringBuilder`; `StringBuffer` only adds `synchronized`.

QUICK CHECK — no marks, just for you.

1. Why is building a string with += inside a loop O(n squared)?

- ◯ Because each iteration allocates a new String and copies every character accumulated so far
- ◯ Because string concatenation always sorts the characters
- ◯ It is not — the compiler converts it to a StringBuilder automatically
- ◯ Because the JVM cannot optimise loops containing string operations

Check answer

2. Is it still accurate to say 'the compiler turns every + into a StringBuilder'?

- ◯ Yes, on every Java version
- ◯ It was true up to Java 8; since Java 9 a single concatenation expression compiles to invokedynamic against StringConcatFactory, which sizes the result and fills it in one

- No — the compiler never optimises string concatenation
- Only when the operands are compile-time constants

Check answer

3. When is StringBuffer the right choice over StringBuilder?

- Whenever the code might one day be multi-threaded
- When building very large strings, because it is faster
- Essentially never — per-method locking costs when unneeded and does not make a SEQUENCE of appends atomic, which is what shared use would actually require
- When the builder is a field rather than a local variable

Check answer

‹ Previous
The String Pool and intern()

Next ›
Wrapper Classes, Autoboxing and the Integer Cache

LEARN MORE

ARTICLES & DOCS

StringBuilder — API docs
docs.oracle.com

VIDEOS



String vs StringBuilder vs StringBuffer | What is Immutability? | Java String Operations | Geekific

Geekific



String vs StringBuilder vs StringBuffer in Java | Immutability & Thread Safety Explained

The Curious Coder

Hand-picked references for this lesson.